

tastemam Catering Web Application

Security Analysis Report

OWASP ZAP 2.17.0

Ata Soytürk-202211063, CENG382

This report was prepared to assess the security status of the tastemam catering web application. The report contains the results of an automated security scan conducted using the OWASP ZAP (Zed Attack Proxy) 2.17.0 tool. The platform includes critical features such as user registration and login, menu creation and management, shopping cart and order processing, payment simulation, a review system, and an admin panel.

- **Scan Method:** Automatic passive scan (Traditional Spider)
- **Scan Date:** May 15, 2026
- **Scan Time:** 4:25:12 PM
- **Scanned URL:** http://localhost:5247
- **Scanning Scope:** All application pages and endpoints
- **Total Number of Detected Alerts:** 10

EXECUTIVE SUMMARY

As a result of the security scan conducted, no high-level critical security vulnerabilities were identified in the application.

The risk distribution is as follows:

- High Risk: 0 alerts
- Medium Risk: 4 alerts — 40%
- Low Risk: 2 alerts — 20%
- Informational: 4 alerts — 40%

Total: 10 alerts

The confidence distribution is as follows:

- High Confidence: 4 warnings — 40%
- Medium Confidence: 5 warnings — 50%
- Low Confidence: 1 warning — 10%

Key findings:

- Missing Content Security Policy header
- Insufficient anti-clickjacking protection
- Lack of Sub Resource Integrity checks for external resources and absence of the X-Content-Type-Options header.

While these findings are of medium to low technical severity, they represent configuration deficiencies that must be addressed in a production environment.

STATISTICAL DATA

The following statistics were obtained during the scan:

- Total number of endpoints: 21
- 2xx (Success) response rate: 71%
- 3xx (Redirect) response rate: 12%
- 4xx (Client Error) response rate: 9%
- 5xx (Server Error) response rate: 7%
- GET request rate: 85%
- POST request rate: 14%
- text/html content rate: 47%
- text/css content rate: 9%
- text/javascript content rate: 9%
- Slow response rate: 14%

METHODOLOGY

OWASP ZAP is an open-source penetration testing tool developed by the Open Web Application Security Project and widely used around the world.

The following procedure was followed during the scanning process:

- First, the application was run in a local development environment.
- Then, using ZAP's Traditional Spider feature, all application pages were automatically crawled and discovered.

- After the spider scan was completed, the passive scanner was activated, and HTTP request/response pairs were analyzed.
- The findings were classified according to risk and confidence levels.

The following techniques were used during the scanning process:

- Passive scanning: Analysis of HTTP headers, verification of security configurations
- Spider scanning: Automatic discovery of all links and forms
- Alert analysis: Mapping each alert to CWE and WASC references

The following risk levels were assessed during the scan: High, Medium, Low and Informational. Confidence levels: User Confirmed, High and Low

IDENTIFIED SECURITY VULNERABILITIES

Content Security Policy (CSP) Header Not Set

- CWE ID: 693 (Protection Mechanism Failure)
- WASC ID: 15
- Affected URL: GET http://localhost:5247/
- Risk Level: Medium
- Confidence Level: High

Content Security Policy (CSP) is an HTTP security header designed to protect web applications against Cross-Site Scripting (XSS) and other code injection attacks. This header explicitly instructs the browser on which sources it is permitted to load content from.

The tastemam application's server responses do not include a Content-Security-Policy header. This means there is no mechanism to prevent the browser from executing malicious scripts if a potential attacker injects them into the page.

Implemented Solution:

The UseContentSecurityPolicy middleware was added to the ASP.NET Core Program.cs file to define allowed content sources. Under this policy, only scripts, styles, and fonts from trusted sources are permitted to be loaded.

References:

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CSP>
- https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html

Missing Anti-clickjacking Header

- CWE ID: 1021 (Improper Restriction of Rendered UI Layers)
- WASC ID: 15
- Affected URL: GET <http://localhost:5247/>
- Risk Level: Medium
- Confidence Level: Medium

A clickjacking attack is a type of attack where an attacker embeds a legitimate web page within an invisible iframe to trick the user into performing malicious actions without their knowledge. For example, while a user may think they are clicking on an innocent-looking button, they may actually be submitting a hidden form running in the background.

Implemented Solution:

The X-Frame-Options: DENY header was added to the Program.cs file to ensure the application cannot be displayed within any iframe.

References:

- <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/X-Frame-Options>
- <https://datatracker.ietf.org/doc/html/rfc7034>

Sub Resource Integrity Attribute Missing

- CWE ID: 345 (Insufficient Verification of Data Authenticity)
- WASC ID: 15
- Affected URL: GET <http://localhost:5247/Home/Index?kategori=Kokteyl>
- Risk Level: Medium
- Confidence Level: High

Sub Resource Integrity (SRI) is a security mechanism used to verify whether JavaScript and CSS files loaded from external CDN servers match the expected content. This mechanism

defines the encrypted hash of the relevant resource as an `integrity` attribute within the HTML and verifies this hash when the browser downloads the file.

Potential consequences of this vulnerability include the distribution of malicious JavaScript to all users via a CDN attack (supply chain attack), the theft of user data, and the compromise of session information.

Implemented Solution:

The `integrity` and `crossorigin` attributes have been added to all CDN links in the `\_Layout.cshtml` file.

References:

https://developer.mozilla.org/en-US/docs/Web/Security/Subresource_Integrity

Malformed X-Frame-Options Setting

- CWE ID: 1021 (Improper Restriction of Rendered UI Layers)
- WASC ID: 15
- Affected URL: GET <http://localhost:5247/Account/Login>
- Risk Level: Medium
- Confidence Level: Medium

The X-Frame-Options header is an HTTP security header used to protect against clickjacking attacks. The valid values for this header are only DENY and SAMEORIGIN. If a different value is entered, the browser ignores this directive and the protection becomes ineffective.

Potential consequences of this vulnerability include the effective disabling of protection against clickjacking attacks and the theft of user credentials by embedding the login page within an iframe.

Implemented Solution:

The value of the X-Frame-Options header has been corrected to DENY.

References:

<https://datatracker.ietf.org/doc/html/rfc7034#section-2.1>

Cross-Domain JavaScript Source File Inclusion

- CWE ID: 829 (Inclusion of Functionality from Untrusted Control Sphere)
- WASC ID: 15
- Affected URL: GET http://localhost:5247/
- Risk Level: Low
- Confidence Level: Medium

The application loads JavaScript files from external domains, such as the Google Maps JavaScript API. These external sources are outside the direct control of the application developer. If these external servers are compromised, malicious JavaScript code could be executed in the user's browser.

Implemented Solution:

Under the CSP policy, script loading is permitted only from the addresses maps.googleapis.com and maps.gstatic.com. The integrity attribute has been added where applicable.

X-Content-Type-Options Header Missing

- CWE ID: 693 (Protection Mechanism Failure)
- WASC ID: 15
- Affected URL: GET http://localhost:5247/css/site.css
- Risk Level: Low
- Confidence Level: Medium

The X-Content-Type-Options: nosniff header prevents the browser from performing MIME type sniffing. MIME type sniffing is a behavior where the browser attempts to guess the file type by examining the file's content, disregarding the content type specified by the server. Without this header, if an attacker serves a malicious JavaScript file as CSS or an image, the browser may analyze the file's actual content and execute it as a script.

Implemented Solution:

Middleware has been added to the Program.cs file to include the X-Content-Type-Options: nosniff header in all responses.

References:

<https://owasp.org/www-community/Security-Headers>

INFORMATION FINDINGS

The findings in this section do not constitute security vulnerabilities and have been reported by ZAP for the purpose of identifying application behavior.

Authentication Request Identified — High Confidence

(POST http://localhost:5247/Account/Login)

ZAP has detected that the application performs an authentication process at the /Account/Login address. This information does not constitute a security vulnerability. The TasteMam application uses the ASP.NET Core Identity framework, and authentication processes are performed using secure, industry-standard methods. Passwords are stored hashed using the bcrypt algorithm.

Session Management Response Identified — High Security

(GET http://localhost:5247/Home/Detail/4)

ZAP has detected that the application uses cookies for session management. The standard cookie-based authentication mechanism of ASP.NET Core Identity is being used. Cookies are configured with the HttpOnly and Secure flags.

Modern Web Application — Medium Security

(GET http://localhost:5247/Home/Index?kategori=Kokteyl)

ZAP has detected that the application uses modern web technologies (JavaScript frameworks, dynamic content). This indicates that traditional spider crawling may not be able to access some dynamic content. The application uses modern libraries such as Bootstrap, Font Awesome, and the Google Maps API.

APPLICATION SECURITY INFRASTRUCTURE ASSESSMENT

In addition to the findings of the security scan, the overall security infrastructure of the Tastemam application has been assessed as follows:

Authentication and Authorization:

- The application uses the ASP.NET Core Identity library.
- User passwords are stored after being hashed using the PBKDF2 algorithm.
- Role-based access control (RBAC) has been implemented, with Admin, Caretaker, and User roles defined.
- Each role can access only the pages within its own scope.

Session Management:

- ASP.NET Core's built-in cookie-based authentication mechanism is used.
- Session cookies are marked as HttpOnly and are inaccessible via JavaScript.

Database Security:

- The use of Entity Framework Core ORM provides natural protection against SQL injection attacks.
- Parameterized queries are automatically applied.

Input Validation:

- Validation is applied to all form fields using Data Annotations.
- Model binding operations are securely handled by ASP.NET Core.

Sensitive Information Management:

- Sensitive information is stored using the .NET User Secrets mechanism and is not committed to the source code repository.